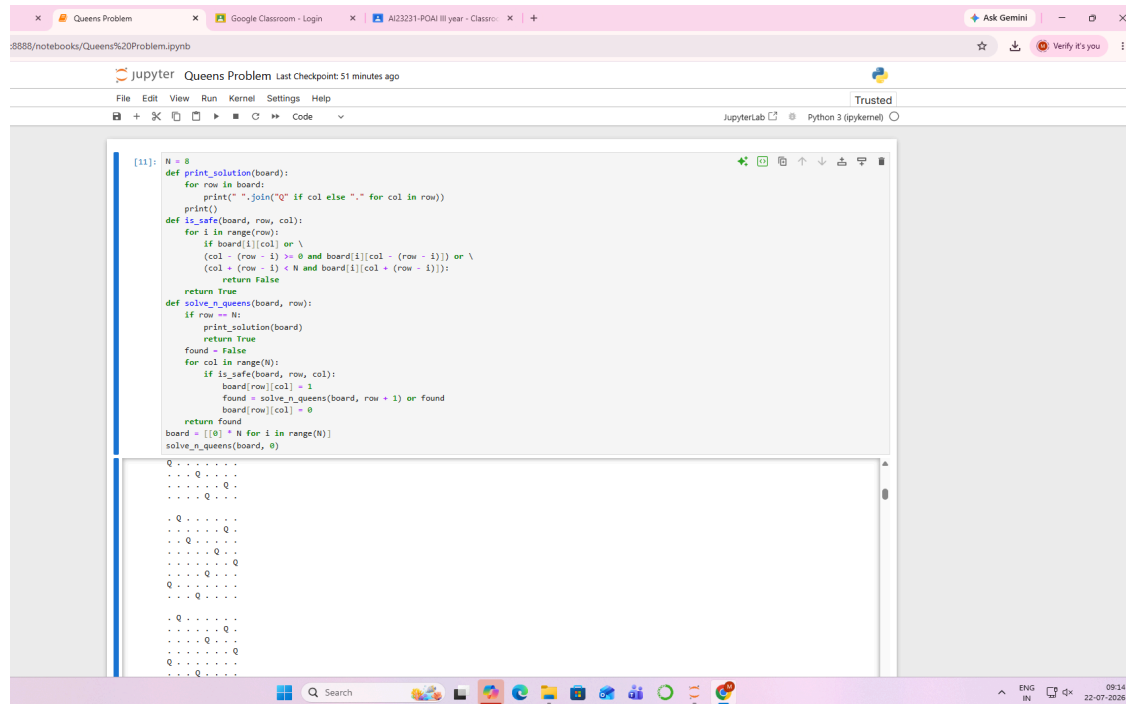


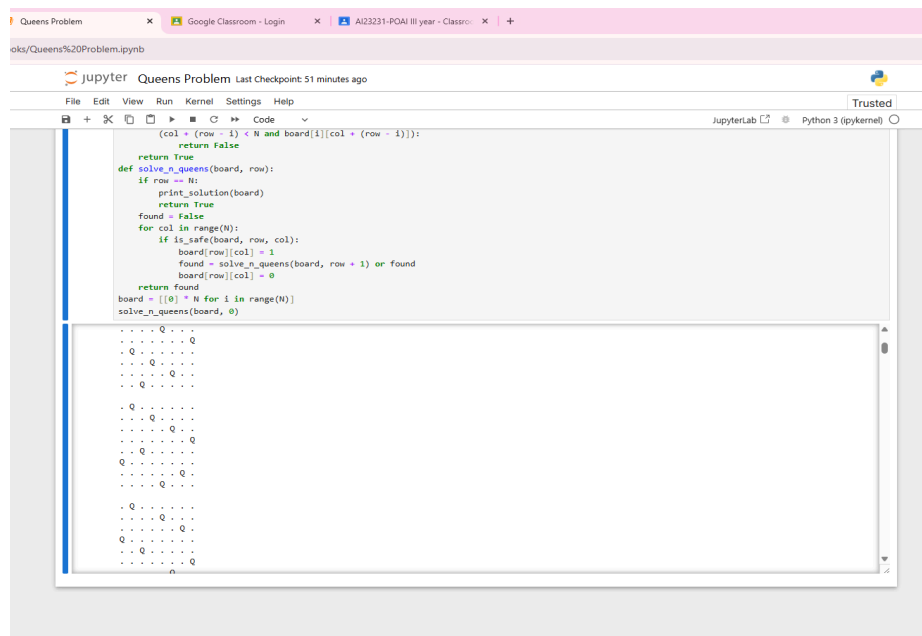
Programs on Problem Solving

a: Solve the 8 Queens Problem



```
[11]: N = 8
def print_solution(board):
    for row in board:
        print(" ".join("Q" if col else "." for col in row))
    print()
def is_safe(board, row, col):
    for i in range(row):
        if board[i][col] or \
            (col - (row - i) >= 0 and board[i][col - (row - i)]) or \
            (col + (row - i) < N and board[i][col + (row - i)]):
            return False
    return True
def solve_n_queens(board, row):
    if row == N:
        print_solution(board)
        return True
    found = False
    for col in range(N):
        if is_safe(board, row, col):
            board[row][col] = 1
            found = solve_n_queens(board, row + 1) or found
            board[row][col] = 0
    return found
board = [[0] * N for i in range(N)]
solve_n_queens(board, 0)

Q . . . . .
. . Q . . .
. . . . Q .
. . . . . Q
. Q . . . .
. . . Q . .
. . . . . Q
. . . . . Q
Q . . . . .
. . . . . Q
. . . . . Q
. . . . . Q
Q . . . . .
. . . . . Q
. . . . . Q
Q . . . . .
. . . . . Q
. . . . . Q
Q . . . . .
```



```
(col + (row - i) < N and board[i][col + (row - i)]):
    return False
return True
def solve_n_queens(board, row):
    if row == N:
        print_solution(board)
        return True
    found = False
    for col in range(N):
        if is_safe(board, row, col):
            board[row][col] = 1
            found = solve_n_queens(board, row + 1) or found
            board[row][col] = 0
    return found
board = [[0] * N for i in range(N)]
solve_n_queens(board, 0)

. . . . . Q
. . . . . Q
. Q . . . .
. . . . Q .
. . . . . Q
. . . . . Q
. . . . . Q
Q . . . . .
. . . . . Q
. . . . . Q
Q . . . . .
. . . . . Q
. . . . . Q
Q . . . . .
. . . . . Q
. . . . . Q
Q . . . . .
```